

ANKARA ÜNİVERSİTESİ

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü



Oğrenci: Ahmet Akgun

Oğrenci No: 22290602

Danışman: Enver Bağcı

BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

Final Rapor

Projeler:

1. Veritabanı Performans Optimizasyonu ve İzleme
2. Veritabanı Yedekleme ve Felaketten Kurtarma Planı
5. Veri Temizleme ve ETL Süreçleri Tasarımı

• GitHub Depo Linki: <https://github.com/aakgunnn>

• Haftalık Video Takip Bağlantısı: <https://www.youtube.com/@ahmetakgun359>

İçindekiler

I Veritabani Performans Optimizasyonu ve Izleme (Proje 1)	3
1 Giriş	3
2 Asama 1: Büyük Veri Ortaminin Hazirlanmasi	3
3 Asama 2: Veritabani Izleme (Monitoring)	3
3.1 Tablo Istatistikleri (pg_stat_user_tables)	3
3.2 Aktif Oturumlar (pg_stat_activity)	4
3.3 Disk Alani Yonetimi	4
4 Asama 3: Indeks Yonetimi	4
4.1 Indeks Oncesi vs Sonrasi Karsilastirma	4
4.2 Olusturulan Indeksler	4
4.3 Gereksiz Indeks Tespiti	4
5 Asama 4: Sorgu Iyilestirme	5
5.1 Alt Sorgu vs JOIN + GROUP BY	5
5.2 SELECT * vs Sadece Gerekli Sutunlar	5
5.3 Fonksiyon Kullanimi ve Index Bypass	5
6 Asama 5: Veri Yoneticisi Roller	5
7 Sonuc	5
II Veritabani Yedekleme ve Felaketten Kurtarma Plani (Proje 2)	6
1 Giriş	6
2 Asama 1: Test Ortaminin Hazirlanmasi	6
3 Asama 2: Yedekleme Stratejileri	6
3.1 Tam Yedekleme (Full Backup)	6
3.2 Sema Yedekleme (Schema-Only / Diferansiyel Temel)	7
3.3 Tablo Bazli Yedekleme (Artik / Incremental)	7
3.4 Sadece Veri Yedegi (Data-Only)	7
3.5 Yedekleme Boyut Karsilastirmasi	7
4 Asama 3: Felaket Senaryolari	7
4.1 Senaryo 1: Toplu Veri Silme (DELETE)	7
4.2 Senaryo 2: Tablo Silme (DROP TABLE)	8
5 Asama 4: Yedekten Kurtarma ve Dogrulama	8
5.1 Yedek Dogrulama (Backup Validation)	8
5.2 Tam Kurtarma (Full Restore)	8

5.3	Tablo Bazli Kurtarma	8
5.4	Kurtarma Sonrasi Dogrulama	8
6	Sonuc	9
III	Veri Temizleme ve ETL Surecleri Tasarimi (Proje 5)	10
1	Giris	10
2	Ham Veri Kaynaklari	10
2.1	Musteri Verisi (raw_data_customers.csv)	10
2.2	Siparis Verisi (raw_data_orders.csv)	10
3	Extract (Veri Cikarma) – stage1_extract.sql	10
4	Transform (Veri Donusturme) – stage2_transform.sql	11
4.1	Email Dogrulama	11
4.2	Telefon Standardizasyonu	11
4.3	Tarih ve Sehir Standardizasyonu	11
4.4	Negatif ve NULL Deger Kontrolu	11
4.5	Duplicate Kontrolu	12
5	Load (Veri Yukleme)	12
6	Veri Kalitesi Raporu – stage3_quality_report.sql	12
7	Sonuc	13

Kısım I

Veritabani Performans Optimizasyonu ve Izleme (Proje 1)

1 Giriş

Bu projede, 1.6 milyon satirlik buyuk bir veritabani uzerinde performans analizi yapilmis ve optimizasyon teknikleri uygulanmistir. Proje bes asamadan olusmaktadır:

1. Buyuk veritabani ortaminin hazirlanmasi (100K musteri, 500K siparis, 1M detay)
2. Veritabani izleme (DMV karsiligi sistem gorunum sorgulamalari)
3. Indeks yonetimi (olusturma, oncesi/sonrasi karsilastirma, gereksiz indeks tespiti)
4. Sorgu iyilestirme (yavas sorgulari analiz etme ve optimize etme)
5. Veri yoneticisi rolleri ve erisim yonetimi

2 Asama 1: Buyuk Veri Ortaminin Hazirlanmasi

PostgreSQL'in `generate_series` fonksiyonu kullanilarak toplu veri uretilmistir:

Tablo	Kayit Sayisi	Boyut
customers	100.000	12 MB
products	1.000	128 KB
orders	500.000	39 MB
order_items	1.000.000	71 MB
Toplam	1.601.000	131 MB

Tablo 1: Veritabani Tablo Istatistikleri

3 Asama 2: Veritabani Izleme (Monitoring)

SQL Server'daki DMV (Dynamic Management Views) ve SQL Profiler araclarinin PostgreSQL karsiliklari kullanilmistir.

3.1 Tablo Istatistikleri (pg_stat_user_tables)

```
1 SELECT relname AS tablo_adi, n_live_tup AS canli_satir,
2     n_dead_tup AS olu_satir, seq_scan AS sirasal_tarama,
3     idx_scan AS indeks_tarama
4 FROM pg_stat_user_tables ORDER BY n_live_tup DESC;
```

Listing 1: DMV Karsiligi Tablo Izleme

3.2 Aktif Oturumlar (pg_stat_activity)

```
1 SELECT pid, username, state, LEFT(query, 80) AS sorgu,
2      NOW() - query_start AS gecen_sure
3 FROM pg_stat_activity WHERE datname = 'perf_demo_db';
```

Listing 2: SQL Profiler Karsiligi

3.3 Disk Alani Yonetimi

```
1 SELECT tablename AS tablo,
2      pg_size_pretty(pg_total_relation_size(tablename::regclass)) AS toplam
3      ,
4      pg_size_pretty(pg_relation_size(tablename::regclass)) AS veri
5 FROM pg_tables WHERE schemaname = 'public';
```

Listing 3: Tablo ve Indeks Boyutlari

4 Asama 3: Indeks Yonetimi

Indeks olusturmadan once ve sonra ayni sorgular EXPLAIN ANALYZE ile calistirilarak performans farklari olculmustur.

4.1 Indeks Oncesi vs Sonrasi Karsilastirma

Test Senaryosu	Oncesi (ms)	Sonrasi (ms)	Iyilesme
Sehre gore musteri arama	15.5	6.1	%61
Tarih araliginda siparis	59.8	12.2	%80
Kategori + fiyat filtresi	0.30	0.18	%40
Buyuk JOIN sorgusu	2.86	0.25	%91

Tablo 2: Indeks Oncesi/Sonrasi Performans Karsilastirmasi

4.2 Olusturulan Indeksler

```
1 CREATE INDEX idx_customers_city_active ON customers(city, is_active);
2 CREATE INDEX idx_orders_date_status ON orders(order_date, status);
3 CREATE INDEX idx_orders_customer_id ON orders(customer_id);
4 CREATE INDEX idx_products_category_price ON products(category, price);
5 CREATE INDEX idx_order_items_order_id ON order_items(order_id);
```

Listing 4: Bilesik Indeks Ornekleri

4.3 Gereksiz Indeks Tespiti

pg_stat_user_indexes gorunumu kullanilarak hic kullanilmayan indeksler tespit edilmiştir:

```

1 SELECT indexrelname, idx_scan AS kullanim,
2    pg_size_pretty(pg_relation_size(indexrelid)) AS boyut,
3    CASE WHEN idx_scan = 0 THEN 'KULLANILMIYOR' END AS durum
4 FROM pg_stat_user_indexes ORDER BY idx_scan ASC;

```

Listing 5: Kullanılmayan Indeks Tespiti

5 Asama 4: Sorgu İyileştirme

Uzun süren sorgular analiz edilerek optimize edilmiştir.

5.1 Alt Sorgu vs JOIN + GROUP BY

N+1 alt sorgu problemi, tek bir LEFT JOIN + GROUP BY ifadesine donusturulerek cozulmustur.

5.2 SELECT * vs Sadece Gerekli Sutunlar

SELECT * yerine sadece ihtiyac duyulan sutunlarin secilmesiyle veri transferi azaltilmis ve sorgu suresi **3.78 ms'den 0.89 ms'ye** dusurulmustur.

5.3 Fonksiyon Kullanimi ve Index Bypass

WHERE UPPER(city) = 'ANKARA' seklindeki fonksiyon kullaniminin indeksi devre disi biraktigi gosterilmistir. Dogrudan WHERE city = 'Ankara' kullanimi ile sorgu suresi **49.8 ms'den 4.8 ms'ye** dusurulmustur (%90 iyilesme).

6 Asama 5: Veri Yoneticisi Roller

Farkli yetki seviyelerine sahip uc rol tanimlanmistir:

Rol	Kullanici	Yetkiler
perf_monitor	monitor_user	Sadece okuma + sistem gorunum erisimi
data_analyst	analyst_user	Okuma + sinirli yazma (tablo olusturma)
dba_admin	admin_user	Tam yetki

Tablo 3: Rol ve Yetki Matrisi

7 Sonuc

Bu projede, 1.6 milyon satirlik buyuk bir veritabani uzerinde kapsamli performans analizi ve optimizasyon calismalari gerceklestirilmistir. Indeks olusturma ile sorgu surelerinde %40 ile %91 arasinda iyilesme saglanmis, yavas sorgular optimize edilmis, sistem izleme mekanizmalari kurulmus ve rol bazli erisim yonetimi uygulanmistir.

Kısım II

Veritabani Yedekleme ve Felaketten Kurtarma Planı (Proje 2)

1 Giriş

Bu projede, bir veritabanının yedekleme ve felaketten kurtarma (Disaster Recovery) planı tasarlanmış ve uygulanmıştır. SQL Server ortamındaki yedekleme stratejileri PostgreSQL mimarisine uyarlanarak dört aşamada gerçekleştirilmiştir:

1. Test veritabanı ortamının hazırlanması
2. Tam, semai, tablo bazlı ve sadece veri yedeklemesi stratejileri
3. Felaket senaryoları (DELETE ve DROP simülasyonu)
4. Yedekten kurtarma ve doğrulama testleri

2 Asama 1: Test Ortamının Hazırlanması

dr_demo_db adında bir veritabanı oluşturulmuş ve company semasi altında üç tablo ile doldurulmuştur:

Tablo	Kayıt	Acıklama
company.employees	10.000	Çalışan bilgileri
company.projects	500	Proje bilgileri
company.customers	5.000	Müşteri bilgileri (kritik veri)

Tablo 4: Test Veritabanı Tabloları

3 Asama 2: Yedekleme Stratejileri

PostgreSQL'in pg_dump aracı kullanılarak dört farklı yedekleme tipi uygulanmıştır.

3.1 Tam Yedekleme (Full Backup)

Tüm veritabanının yapısal ve verisel olarak eksiksiz yedeği:

```
1 pg_dump -U postgres -d dr_demo_db -F c -f full_backup.backup
```

Listing 6: Tam Yedekleme Komutu

-F c parametresi ile sıkıştırılmış (Custom) format kullanılarak disk alanı optimize edilmiştir. Sonuç: **207 KB** boyutunda eksiksiz yedek.

3.2 Sema Yedekleme (Schema-Only / Diferansiyel Temel)

Sadece tablo yapilari, indeksler ve kisitlarin yedegi:

```
1 pg_dump -U postgres -d dr_demo_db --schema-only -f schema_only.sql
```

Listing 7: Sema Yedekleme

Sonuc: **4.48 KB**. Fark (Differential) yedeklemenin temeli olarak kullanilir.

3.3 Tablo Bazli Yedekleme (Artik / Incremental)

Sadece belirli tablolarin bagimsiz yedegi:

```
1 pg_dump -U postgres -d dr_demo_db -t company.employees -F c -f  
  table_employees.backup  
2 pg_dump -U postgres -d dr_demo_db -t company.customers -F c -f  
  table_customers.backup
```

Listing 8: Tablo Bazli Yedekleme

Bu strateji sayesinde felaket aninda tum veritabanini geri yuklemek yerine sadece hasar goren tablo kurtarilabilir.

3.4 Sadece Veri Yedegi (Data-Only)

Yapi haric sadece INSERT ifadelerini icerir:

```
1 pg_dump -U postgres -d dr_demo_db --data-only -f data_only.sql
```

Listing 9: Data-Only Yedekleme

Sonuc: **722 KB**. Mevcut yapiya veri aktarimi icin idealdir.

3.5 Yedekleme Boyut Karsilastirmasi

Yedek Tipi	Boyut	Kullanım Alanı
Tam (Full, -F c)	207 KB	Tum veritabani kurtarma
Sema (Schema-Only)	4.48 KB	Yapi yeniden olusturma
Tablo Bazli	208 KB	Tek tablo kurtarma
Sadece Veri (Data-Only)	722 KB	Veri aktarimi

Tablo 5: Yedekleme Tiplerine Gore Boyut Karsilastirmasi

4 Asama 3: Felaket Senaryolari

Gercek dunyadaki kazalari simule etmek icin iki felaket senaryosu uygulanmistir.

4.1 Senaryo 1: Toplu Veri Silme (DELETE)

IT departmanindaki 2.000 calisanin yanlislikla silinmesi:

```
1 -- !!! 2000 kayit siliniyor !!!  
2 DELETE FROM company.employees WHERE department = 'IT';  
3 -- Sonuc: 10.000 -> 8.000 kayit (2.000 kayip)
```

Listing 10: Felaket Senaryosu 1

4.2 Senaryo 2: Tablo Silme (DROP TABLE)

Kritik musteriler tablosunun tamamen yok edilmesi:

```
1 -- !!! Tablo tamamen siliniyor !!!
2 DROP TABLE company.customers CASCADE;
3 -- Sonuc: Tablo ve 5.000 kayıt tamamen kayboldu
```

Listing 11: Felaket Senaryosu 2

5 Asama 4: Yedekten Kurtarma ve Dogrulama

5.1 Yedek Dogrulama (Backup Validation)

Kurtarma islemine baslamadan once yedegin bozuk olmadigini dogrulamak kritiktir:

```
1 pg_restore --list full_backup.backup
2 -- Sonuc: [BASARILI] 3 tablo verisi tespit edildi
```

Listing 12: Yedek Butunluk Testi

5.2 Tam Kurtarma (Full Restore)

Veritabanı sıfırdan yeniden olusturularak tam yedekten geri yuklendi:

```
1 dropdb -U postgres dr_demo_db
2 createdb -U postgres dr_demo_db
3 pg_restore -U postgres -d dr_demo_db full_backup.backup
```

Listing 13: Full Restore

Geri yukleme suresi: **0.55 saniye**. Tum 3 tablo eksiksiz kurtarildi.

5.3 Tablo Bazli Kurtarma

Sadece customers tablosu kasten silinip, tablo bazli yedekten bagimsiz olarak geri yuklendi:

```
1 pg_restore -U postgres -d dr_demo_db table_company_customers.backup
2 -- Sonuc: 5.000 musteriler kaydi basariyla kurtarildi
```

Listing 14: Tablo Bazli Restore

5.4 Kurtarma Sonrasi Dogrulama

Tablo	Felaket Oncesi	Felaket Sonrasi	Kurtarma Sonrasi
employees	10.000	8.000	10.000
projects	500	500	500
customers	5.000	YOK (DROP)	5.000

Tablo 6: Felaket Oncesi / Sonrasi / Kurtarma Karsilastirmasi

6 Sonuc

Bu projede, bir veritabaninin felaketten kurtarma plani basariyla tasarlanmis ve test edilmistir. Dort farkli yedekleme stratejisi (tam, sema, tablo bazli, sadece veri) uygulanmis, gercekci felaket senaryolari simule edilmiş ve tum veriler yedeklerden eksiksiz olarak kurtarilmistir. Yedek dosyalarinin butunlugu `pg_restore --list` komutu ile dogrulanmistir.

Kısım III

Veri Temizleme ve ETL Surecleri Tasarimi (Proje 5)

1 Giriş

Bu proje kapsamında, büyük veri kumelerinin temizlenmesi ve işlenmesi için bir ETL (Extract, Transform, Load) pipeline'i PostgreSQL üzerinde tamamen SQL tabanlı olarak tasarlanmış ve uygulanmıştır. Pipeline üç SQL script dosyasından oluşmaktadır:

1. **stage1_extract.sql**: Ham CSV dosyalarından veriyi staging tablolarına yükler.
2. **stage2_transform.sql**: Veriyi temizler, doğrular, standartlaştırır ve hedef tablolara yükler.
3. **stage3_quality_report.sql**: Veri kalitesi raporları oluşturur.

2 Ham Veri Kaynakları

Projede iki adet ham CSV dosyası kullanılmıştır. Bu dosyalar kasıtlı olarak çeşitli veri kalitesi sorunları içerecek şekilde tasarlanmıştır:

2.1 Müşteri Verisi (raw_data_customers.csv)

13 satırlık bu dosyada: eksik isim alanları, tutarsız email formatları, farklı telefon ve tarih formatları, tutarsız şehir adları, negatif/NULL tutarlar, duplicate kayıtlar ve fazla boşluklar bulunmaktadır.

2.2 Sipariş Verisi (raw_data_orders.csv)

12 satırlık bu dosyada: negatif/sıfır miktar değerleri, sayısal olmayan miktar (abc), eksik fiyat bilgileri, sistemde olmayan müşteri ID'leri, tutarsız sipariş durumları ve tekrarlanan siparişler bulunmaktadır.

3 Extract (Veri Çıkarma) – stage1_extract.sql

Pipeline'in ilk aşamasında, ham veriler tüm sütunları VARCHAR tipinde tutan **staging tablolarına** yüklenmiştir. Böylece henüz doğrulanmamış verilerin tip hataları oluşması engellenmiştir.

```
1 CREATE TABLE staging_customers (  
2     id VARCHAR(10),  
3     full_name VARCHAR(200),  
4     email VARCHAR(200),  
5     phone VARCHAR(50),  
6     city VARCHAR(100),  
7     registration_date VARCHAR(50),
```

```

8      total_spent VARCHAR(50)
9  );
10
11 \copy staging_customers FROM 'raw_data_customers.csv' WITH (FORMAT csv,
    HEADER true);

```

Listing 15: Staging Tabloları ve Veri Yükleme

4 Transform (Veri Donusturma) – stage2_transform.sql

Donusturma asamasi, pipeline'in en kapsamli bolumudur. Tum temizleme ve standartlas-tirma islemleri tek bir SQL INSERT INTO ... SELECT ifadesi icinde gercekleştirilmiştir.

4.1 Email Dogrulama

Regex tabanlı email dogrulaması ile gecersiz formattaki adresler (elif_celik@company, can.polat@) tespit edilerek ilgili kayıtlar atlanmıştır:

```

1 WHERE TRIM(email) ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'

```

Listing 16: Regex ile Email Dogrulama

4.2 Telefon Standardizasyonu

Farklı formatlardaki telefon numaraları (05551234568 vs +90 555 123 4567) SQL CASE ifadeleri ve REGEXP_REPLACE fonksiyonu ile tek bir standart formata donusturulmuştur:

```

1 CASE
2   WHEN LENGTH(REGEXP_REPLACE(phone, '\D', '', 'g')) = 11
3     THEN '+90 ' || SUBSTRING(REGEXP_REPLACE(phone, '\D', '', 'g'), 2,
4       3)
5         || ' ' || SUBSTRING(..., 5, 3)
6         || ' ' || SUBSTRING(..., 8, 4)
7   ...
8 END AS phone

```

Listing 17: Telefon Format Donusumu

4.3 Tarih ve Sehir Standardizasyonu

YYYY-MM-DD, DD/MM/YYYY ve YYYY/MM/DD gibi farklı tarih formatları regex pattern eslesi-miyle otomatik algılanıp TO_DATE fonksiyonu ile ISO 8601 standardına çevrilmiştir. Sehir adları INITCAP() fonksiyonu ile standartlaştırılmıştır (ankara → Ankara).

4.4 Negatif ve NULL Deger Kontrolu

```

1 GREATEST(
2   COALESCE(
3     CASE
4       WHEN UPPER(TRIM(total_spent)) = 'NULL' OR TRIM(total_spent)
5       = '' THEN 0
6       ELSE CAST(total_spent AS NUMERIC(12,2))
7     END, 0

```

```

7      ), 0
8 ) AS total_spent

```

Listing 18: Negatif/NULL Tutar Duzeltme

4.5 Duplicate Kontrolu

DISTINCT ON ifadesi ile ayni email adresine sahip tekrarlanan kayitlardan sadece ilki alınmistir:

```

1 SELECT DISTINCT ON (LOWER(TRIM(email))) *
2 FROM staging_customers
3 ORDER BY LOWER(TRIM(email)), id::INTEGER

```

Listing 19: Duplicate Eleme

5 Load (Veri Yukleme)

Temizlenmis veriler, referans butunlugu (REFERENCES) ve is kurallari (CHECK) kisit-
larina sahip hedef tablolara yuklenmistir:

```

1 CREATE TABLE clean_customers (
2     customer_id SERIAL PRIMARY KEY,
3     full_name VARCHAR(100) NOT NULL,
4     email VARCHAR(100) UNIQUE NOT NULL,
5     phone VARCHAR(20),
6     city VARCHAR(50),
7     registration_date DATE,
8     total_spent NUMERIC(12,2) DEFAULT 0.00
9 );
10
11 CREATE TABLE clean_orders (
12     order_id INTEGER PRIMARY KEY,
13     customer_id INTEGER REFERENCES clean_customers(customer_id),
14     quantity INTEGER CHECK (quantity > 0),
15     ...
16 );

```

Listing 20: Hedef Tablo Semalari

6 Veri Kalitesi Raporu – stage3_quality_report.sql

Son asamada, surece dair kapsamli bir denetim raporu SQL sorgulari ile olusturulmustur.
Rapor su bilgileri icermektedir:

- Ham vs. temiz kayıt sayisi karsilastirmasi
- Atlanan musterilerin detayli listesi (neden + hangi satir)
- Atlanan siparislerin detayli listesi
- Donusturme detaylari (negatif tutar, NULL deger duzeltmeleri)
- Temiz veri dogrulama (tum temiz kayitlarin listesi)

- Sehir bazli musteri dagilimi
- Durum bazli siparis ve ciro istatistikleri

1	metrik		deger
2	-----+-----		
3	Ham Musteri Sayisi		13
4	Temiz Musteri Sayisi		9
5	Ham Siparis Sayisi		12
6	Temiz Siparis Sayisi		6
7			
8	metrik		deger
9	-----+-----		
10	TOPLAM CIRO (Tamamlandi)		18,000.00 TL

Listing 21: Rapor Ciktisi (Ozet)

7 Sonuc

Bu projede, gercek dunya senaryolarina uygun bir ETL pipeline'i tamamen SQL tabanlı olarak tasarlanmış ve PostgreSQL üzerinde basariyla uygulanmistir. Regex (~), CASE, COALESCE, GREATEST, INITCAP, DISTINCT ON, TO_DATE ve REGEXP_REPLACE gibi guclu SQL fonksiyonlari kullanilarak veri temizleme, dogrulama, standartlastirma ve raporlama islemleri gercekleştirilmiştir.